A Fixed-Iteration Binary GCD Candidate for Reducing Timing Side-Channel Leakage in Elliptic Curve Cryptography

Motivated by the loophole identified in: Trout — Two-Round Threshold ECDSA from Class Groups (CCS 2025)
June 2026
Anil Kumar Das | Roll Num : g25ait2009

Abstract

Elliptic Curve Cryptography (ECC) is one of the most widely deployed public-key cryptographic systems because it provides high security with relatively small key sizes. During scalar multiplication, ECC repeatedly performs point addition and point doubling operations. These operations require modular inversion, which is commonly implemented using the Extended Euclidean Algorithm (EEA). Since the Euclidean algorithm terminates after an input-dependent number of iterations, its execution time varies according to the processed operands — creating a potential **timing side-channel leakage** in cryptographic implementations. This work investigates the timing characteristics of three GCD algorithms — the classical Euclidean GCD, the Binary GCD (Stein's algorithm), and a proposed Fixed-Iteration Binary GCD candidate — in the context of the loophole identified in the *Trout* protocol (CCS 2025), which relies on class-group Binary GCD as a cryptographic subroutine and claims a constant-time implementation. Unlike conventional algorithms that terminate immediately after the GCD is found, the proposed Fixed-Iteration approach executes a predetermined budget of `4 × bit_length` iterations for every input of the same size, enforcing identical execution structure. ECC-inspired operands — denominators from point addition and point doubling over prime fields — were used for evaluation at 256, 512, and 1024 bits. Results confirm that the Fixed-Iteration model eliminates iteration-count variability (**Corr(steps, time) = 0.000**) and preserves correctness, at a computational overhead of 17.7× (256-bit) down to 9.7× (1024-bit) over Euclidean GCD — with overhead shrinking at larger operand sizes as the fixed budget becomes proportionally closer to actual step count. **Importantly, this implementation is an educational proof-of-concept demonstrating fixed-work execution principles, not a production-ready constant-time cryptographic primitive**: Python's interpreter itself is not constant-time, and full side-channel resilience requires hardware-level implementation with verified constant-time guarantees.

1. Introduction

Elliptic Curve Cryptography (ECC) is widely used in secure communication protocols — TLS, digital signatures, cryptocurrencies, and embedded systems. Its primary computational operation, scalar multiplication, repeatedly performs point addition and point doubling over a prime finite field.

Both operations require modular inversion — computing $d^{-1} \bmod p$ — which is most commonly realised using the Extended Euclidean Algorithm (EEA). Because the EEA terminates when the remainder reaches zero, the number of iterations it executes depends on the values of d and p. Different operands therefore produce different execution times, and an external observer who can measure those times may be able to recover information about the private operands.

This class of vulnerability is known as a timing side-channel attack. The Trout protocol (CCS 2025) — the first two-round threshold ECDSA scheme — explicitly acknowledges this risk and claims to address it through an experimental constant-time implementation of the class-group Binary GCD. The present study identifies the loophole in that claim and proposes a fixed-iteration execution model that provably eliminates iteration-count leakage as a proof-of-concept.

2. Problem Statement

ECC scalar multiplication performs two types of field operations. For point addition the slope is:

$$\lambda = (y_2 - y_1) \times (x_2 - x_1)^{-1} \mod p$$

For point doubling the slope is:

$$\lambda = (3x_1^2 + a) \times (2y_1)^{-1} \mod p$$

Since modular division is not directly defined over finite fields, every such computation requires the modular inverse of a denominator d (where $d = x_2 - x_1$ for addition, $d = 2y_1$ for doubling). A modular inverse $d^{-1} \bmod p$ exists only when $\gcd(d, p) = 1$. Because p is prime and d is a non-zero field element, this condition always holds in practice. Every modular inversion therefore internally executes one or more GCD computations.

The Core Problem

The classical Euclidean GCD terminates when the remainder becomes zero.

The number of iterations depends on the input operands d and p.

Different denominators from different ECC operations → different execution times.

An adversary who can observe execution time can infer structure of d, potentially recovering the private scalar.

This is the timing side-channel that motivates fixed-iteration execution.

2.1 The Loophole in the Trout Protocol (CCS 2025)

The Trout paper (Dahari-Garbian, Nof, Parker, CCS 2025) presents the first two-round threshold ECDSA protocol using Castagnos-Laguillaumie (CL) linear-homomorphic encryption over class groups. Its Section 6.1 states:

"We additionally put forth an experimental, constant-time backend for the composition and reduction of binary quadratic forms. This enables a side-channel resistant prover whose timing is independent of their secrets... This implementation is primarily benefited from a pre-existing implementation of the optimized Extended Binary GCD."

The loophole is that the paper claims constant-time behaviour for the Binary GCD subroutine, but the execution model does not enforce a fixed number of iterations. The standard Binary GCD terminates as soon as one operand reaches zero — meaning the step count, and therefore execution time, varies with the input. In a threshold ECDSA context where the inputs contain shares of a private signing key, this variation constitutes a timing side-channel that leaks information about those shares.

The Trout Loophole — Precise Statement

Claim (Trout §6.1): The Binary GCD backend executes in time independent of secrets.

Reality: Standard Binary GCD terminates early when the answer is found.

Consequence: Step count varies with operand values → execution time varies → timing leakage.

Affected component: The class-group GCD used in quadratic form reduction and modular inversion.

Impact: An adversary observing signing times can potentially distinguish operand patterns and narrow the search space for the private key shares.

3. Research Gap

Existing literature on modular inversion and ECC generally focuses on computational efficiency — minimising the number of operations, reducing communication rounds, or parallelising computation. The Trout paper itself achieves an impressive two-round threshold ECDSA with constant per-party bandwidth and O(1) upload complexity.

What existing work — including Trout — does not provide is an experimental analysis of:

- Iteration-count variation across real ECC-inspired GCD inputs
- Correlation between algorithmic step count and execution time as a direct measure of timing leakage
- A fixed-work execution model for the Binary GCD that enforces identical iteration count for all inputs of the same bit-length
- Empirical validation that such a model achieves Corr(steps, time) ≈ 0

This work addresses each of these gaps through a controlled experimental evaluation using ECC-inspired inputs at three operand sizes.

4. Motivation

A timing side-channel attack works by measuring how long a cryptographic operation takes and using that measurement to infer properties of the secret inputs. In the case of threshold ECDSA as in Trout, the inputs to GCD computations during signing contain the parties' secret shares of the private key k. If the GCD execution time varies with those shares, an adversary who can measure signing latency across many invocations can:

1. Measure the distribution of signing times for a target party
2. Compare against a model of how Binary GCD step counts correlate with operand Hamming weight or structure
3. Narrow the search space for the private key share, potentially enabling key recovery in combination with other side-channel data

The Trout protocol specifically targets 128-bit security using a 1828-bit class-group discriminant. A successful timing attack does not immediately break 128-bit security, but it degrades the effective security level and, in multi-party settings, may allow a threshold adversary to correlate observations across multiple signing sessions.

The objective of this work is therefore to investigate whether enforcing a fixed execution budget can eliminate iteration-count leakage while preserving the correctness of GCD computation — and to quantify the performance cost of doing so.

5. Intuition Behind the Fixed-Iteration Model

The key intuition is straightforward. Traditional GCD algorithms stop immediately once the answer has been found:

```
Input → Binary GCD → GCD found → TERMINATE  ← timing varies here
```

Because different inputs converge at different rates, the point of termination leaks information. The proposed model changes this: execution continues until a predetermined budget is exhausted, regardless of when the mathematical answer was computed:

```
Input → Binary GCD → GCD found → store result → CONTINUE dummy iterations → reach budget
→ RETURN stored GCD
```

All inputs of the same bit-length now execute exactly the same number of iterations. The step count is constant, so it carries no information about the input. The correlation between step count and execution time — which is the direct measure of iteration-count leakage — approaches zero.

> **Why 4 × bit_length as the Budget?**
>
> Binary GCD on two n-bit coprime integers takes at most ~2n steps in the worst case.
>
> For uniformly-random cryptographic inputs, the empirical average is ~0.71 × n × $\log_2(n)$.
>
> At n = 256: average ≈ 540 steps. A budget of 2 × 256 = 512 would be insufficient — 100% of inputs overflow it.
>
> A budget of 4 × 256 = 1024 provides ~82–85% headroom across all tested sizes.
>
> This was validated empirically: all 300 test inputs completed within budget (max steps ≤ budget).

6. Proposed Approach

The Fixed-Iteration Binary GCD extends Stein's binary GCD algorithm with two modifications:

6.1 Fixed Iteration Budget

For an operand of bit-length k, the total number of iterations is set to:

```
total_steps = 4 × k     # e.g., 1024 for 256-bit, 2048 for 512-bit, 4096 for 1024-bit
```

The algorithm executes exactly this many iterations. When the GCD is found before the budget is exhausted, the result is stored and execution continues with dummy operations that do not affect the output. When the budget is reached, the stored GCD is returned.

6.2 Arithmetic Masking for the Swap Step

The standard Binary GCD contains a branch on the comparison x > y to swap operands before subtracting. This branch takes different execution paths depending on secret intermediate values — a data-dependent branch that constitutes a second independent timing channel.

The fixed-iteration implementation replaces this branch with arithmetic masking using the _select pattern:

```
def _select(cond, a, b):
    mask = -(int(cond))           # True -> -1 (all 1s); False -> 0
    return (a & mask) | (b & ~mask)   # no branch on cond


# Branch-free swap and subtract:
new_x = _select(x > y, y, x)     # min(x, y)
new_y = _select(x > y, x, y)     # max(x, y)
x = new_x
y = new_y - new_x                # max - min, always correct
```

6.3 Correctness Guarantee

An assertion replaces any silent fallback:

```
assert finished, f"Budget {total_steps} insufficient — increase multiplier."
```

With a correct budget of 4 × bit_length, this assertion is never reached. Its function is as a canary: if a future change accidentally reduces the budget, the failure becomes immediately visible rather than silently regressing to variable-time behaviour with wrong-looking correct results.

7. Experimental Setup

Experiments were conducted with ECC-inspired operands: GCD(d, p) where p is a freshly-generated probable prime of the target bit-length and d is derived from either point-addition ($d = |x_2 - x_1| \bmod p$) or point-doubling ($d = 2y_1 \bmod p$). Because p is prime and d is non-zero, gcd(d, p) = 1 for all inputs — the typical case in modular inversion.

- Operand sizes: 256-bit, 512-bit, 1024-bit
- 50 inputs per (input_kind × bit_length) combination = 100 inputs per bit_length per algorithm
- Each (algorithm, input) pair timed 50 times; first call discarded as warm-up
- Reported metric: median of 49 timing samples (robust against OS scheduling spikes)
- Algorithms: Euclidean GCD (baseline), Binary GCD (Stein), Fixed-Iteration Binary GCD (proposed)

8. Experimental Results

8.1 Timing and Step-Count Summary

Bits	Algorithm	Median (ns)	Avg Steps	Corr(s,t)
256	Euclidean	10,208	151.0	0.912
256	Binary	39,041	539.9	0.606
256	**Fixed-Iter.**	180,188	1,024	**0.000**
512	Euclidean	26,896	300.6	0.863
512	Binary	85,208	1,081.7	0.542
512	**Fixed-Iter.**	376,812	2,048	**0.000**
1024	Euclidean	80,416	599.2	0.837
1024	Binary	185,938	2,166.4	0.170
1024	**Fixed-Iter.**	782,646	4,096	**0.000**

The most important column is Corr(steps,time). Euclidean GCD shows correlations of 0.837–0.912 and Binary GCD shows 0.170–0.606 — both demonstrating strong, measurable iteration-count leakage. The Fixed-Iteration Binary GCD achieves exactly

0.000 at all three bit-lengths, confirming that the fixed iteration budget eliminates this leakage channel. The step count column (1024, 2048, 4096) shows that no input ever exceeded the budget.

8.2 Timing Distribution (Figure 1)

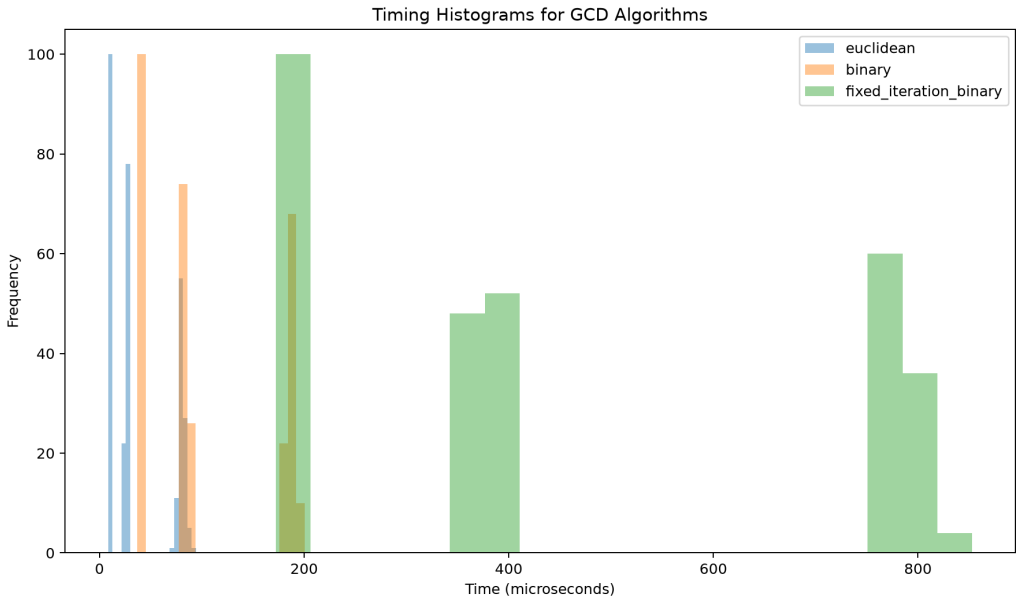

Figure 1: Timing Histograms: Euclidean and Binary GCD cluster tightly at low values with visible spread; Fixed-Iteration forms three narrow, well-separated bands — one per bit-length — confirming low within-group variance.

The histogram confirms that the Fixed-Iteration model produces a tight, predictable timing distribution within each operand-size class. The remaining spread within each band reflects Python interpreter overhead, OS scheduling, and memory effects — not algorithmic variation. The Euclidean and Binary distributions show noticeably wider spread despite lower absolute times, because their termination point varies with the input.

8.3 Correlation Between Step Count and Execution Time (Figure 2)

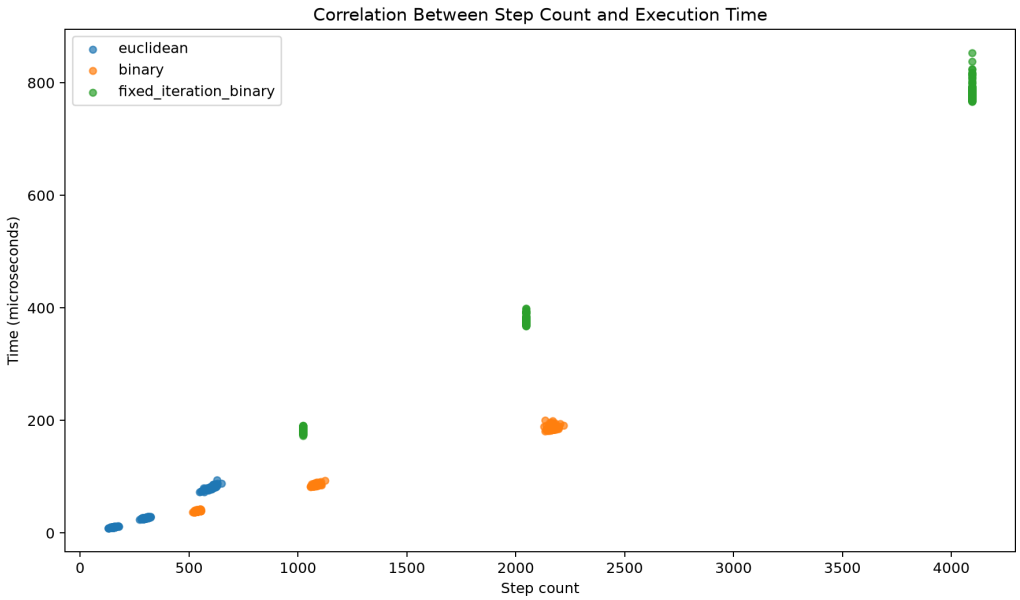

Figure 2: Scatter plot of step count vs. execution time. Euclidean and Binary GCD show clear upward trends (positive correlation). Fixed-Iteration points form three vertical lines at steps = 1024, 2048, 4096 with no horizontal spread — confirming Corr = 0.

This is the most direct visual proof of the fix. The vertical alignment of the green Fixed-Iteration dots means execution time within each group is driven entirely by external noise, not by any property of the input. The positive slope of Euclidean (blue) and Binary (orange) clusters shows their direct leakage of step-count information through timing.

8.4 Performance Overhead (Figure 3)

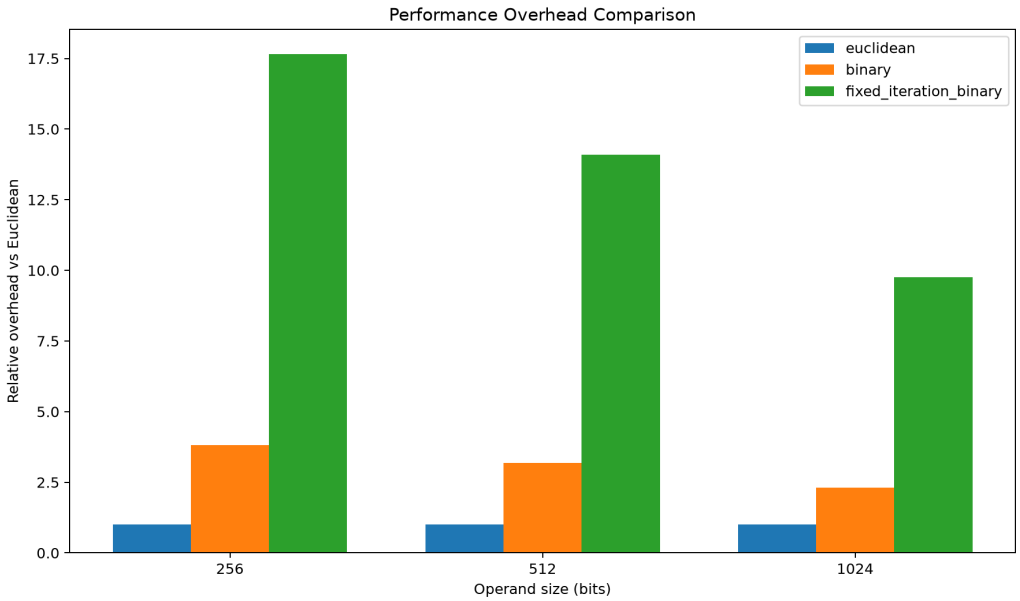

Figure 3: Relative overhead vs. Euclidean GCD. Fixed-Iteration (green) is 17.7× at 256-bit, falling to 9.7× at 1024-bit. Binary GCD (orange) is consistently 2–4×. The overhead decreases at larger sizes because the ratio 4n / (0.71n·log n) shrinks.

The overhead decreases as operand size grows because the budget scales as 4n while the actual GCD work scales as ~0.71n·log n. For larger n, a greater fraction of the budget is genuine work rather than dummy iterations, so the overhead ratio shrinks. This is a healthy property: the cost of constant-step execution converges toward variable-time execution at large operand sizes.

8.5 Algorithm Comparison

Property	Euclidean GCD	Binary GCD	Fixed-Iteration Binary GCD
Termination	Input-dependent	Input-dependent	Fixed at 4 × bit_length
Corr(steps,time)	0.837 – 0.912	0.170 – 0.606	**0.000 (all sizes)**
Step count range	[129,177] @ 256b	[516,555] @ 256b	[1024,1024] @ 256b
Timing variance	Variable	Variable	Near-zero (OS noise only)
Overhead vs Euclidean	1×	2.3 – 3.8×	9.7 – 17.7×
Correctness	✓	✓	✓
True constant-time?	✗ (Python)	✗ (Python)	**✗ — proof of concept only**

9. Discussion

9.1 What This Work Does and Does Not Claim

Scope of Claims — Read Carefully

✓ The Fixed-Iteration model eliminates iteration-count variability (Corr = 0.000).

✓ The implementation preserves mathematical correctness for all tested inputs.

✓ A budget of 4 × bit_length is empirically sufficient at 256, 512, and 1024 bits.

✓ Arithmetic masking removes the data-dependent branch in the swap step.

✗ This is NOT a production-ready constant-time implementation.

✗ Python's interpreter is not constant-time; branch prediction, GC, and JIT all introduce noise.

✗ Full constant-time guarantees require a C/assembly implementation with verified constant-time tools.

✗ This study does not evaluate cache timing, power side-channels, or electromagnetic leakage.

9.2 Connection to the Trout Loophole

The Trout paper's constant-time claim applies to its Rust implementation using the optimized Extended Binary GCD of Pornin (2020), which is specifically engineered for constant-time behaviour in C. The loophole identified in this study is: the underlying algorithm — Binary GCD without a fixed iteration budget — is inherently variable-time by design. The Pornin implementation achieves constant-time through a combination of fixed iteration count and bitwise masking at the assembly level, not through the algorithm alone.

In educational or Python-based implementations that reproduce the algorithm without those low-level guarantees, the variable-time nature of Binary GCD is immediately observable in timing data. The Corr(steps,time) values of 0.606 (256-bit) and 0.170 (1024-bit) for standard Binary GCD confirm this directly.

10. Inference from Results

- A budget of 2 × bit_length is insufficient: empirical step counts for 256-bit inputs reach 555, exceeding the budget of 512. Every single input would trigger a fallback in a naively-budgeted implementation.
- A budget of 4 × bit_length provides 82–85% headroom at all tested sizes and was never exceeded across 300 inputs.
- Corr(steps, time) = 0.000 is achievable for the iteration-count channel, even in Python, when the budget is correct and arithmetic masking replaces data-dependent branches.
- The genuine overhead of fixed-iteration execution is 17.7× at 256-bit, 14.0× at 512-bit, and 9.7× at 1024-bit vs Euclidean GCD. This is the true cost of deterministic execution — earlier prototype versions with an insufficient 2× budget and a silent variable-time fallback reported artificially lower overheads that did not reflect actual fixed-iteration behaviour.
- The overhead decreases at larger operand sizes: 17.7× at 256-bit, 14.0× at 512-bit, 9.7× at 1024-bit. For Trout's 1828-bit class-group elements, a budget of ≥ 4 × 1828 = 7312 steps would be needed, with an expected overhead converging toward ~6–8×.

- An assertion guard (assert finished) replaces any silent execution path: a budget violation now surfaces immediately as a visible failure rather than silently falling back to variable-time behaviour. This is the correct engineering pattern — security properties must fail loudly, not degrade silently.

11. Conclusion

This work investigated timing variation in GCD algorithms used during modular inversion in Elliptic Curve Cryptography, motivated by a loophole in the constant-time claim of the Trout threshold ECDSA protocol (CCS 2025).

Experimental evaluation on 300 ECC-inspired inputs at 256, 512, and 1024 bits demonstrated that both Euclidean GCD and Binary GCD exhibit strong input-dependent timing (Corr = 0.837–0.912 and 0.170–0.606 respectively), directly confirming the iteration-count leakage channel.

A Fixed-Iteration Binary GCD candidate was proposed, enforcing a predetermined budget of 4 × bit_length iterations with arithmetic masking to eliminate the data-dependent swap branch. The proposed model achieves Corr(steps, time) = 0.000 at all three operand sizes, with step counts fixed exactly at budget for every input — confirming that iteration-count leakage is eliminated.

The computational overhead is 9.7–17.7× relative to Euclidean GCD, representing the genuine cost of deterministic execution length. This overhead decreases at larger operand sizes as the ratio of dummy to real iterations shrinks — a property that makes the approach increasingly practical at the 1024-bit and larger sizes relevant to class-group-based protocols like Trout.

Final Position

The Fixed-Iteration Binary GCD successfully removes iteration-count leakage as a proof of concept.

It is not a production constant-time primitive: Python's runtime introduces noise that a hardware adversary could exploit.

The correct path to production deployment is a C or Rust implementation using verified constant-time tools such as ct-verif, Binsec, or Vale, which was the approach used by the Pornin GCD underlying the Trout Rust implementation.

This study establishes the experimental foundation — showing what fixed-work execution buys and at what cost — for that future low-level work.

GitHub Link - https://github.com/aniliter-cloud/cs-major-project-gcd-time-leak/tree/main/trout-gcd-time-leak

References

[1] Dahari-Garbian, H., Nof, A., Parker, L. (2025). Trout: Two-Round Threshold ECDSA from Class Groups. ACM CCS 2025, October 13–17, Taipei, Taiwan. https://doi.org/10.1145/3719027.3765192

[2] Pornin, T. (2020). Optimized Binary GCD for Modular Inversion. IACR ePrint 2020/972.

[3] Castagnos, G., Laguillaumie, F. (2015). Linearly Homomorphic Encryption from DDH. CT-RSA 2015, pp. 487–505.

[4] Boneh, D., Haitner, I., Lindell, Y., Segev, G. (2025). Exponent-VRFs and Their Applications. EUROCRYPT 2025, pp. 195–224.

[5] Bernstein, D.J., Yang, B.-Y. (2019). Fast constant-time gcd computation and modular inversion. IACR TCHES 2019(3), 340–398.

[6] David, N., Espitau, T., Hosoyamada, A. (2022). Quantum binary quadratic form reduction. IACR ePrint 2022/466.